Spring Batch 톺아보기

우아한테크코스 7기 BE 돔푸

목차

- 배치의 필요성
- Spring Batch의 주요 컴포넌트
- Job/Step을 구성하는 방법
- 배치 프로그램을 설계해보자

배치의 필요성

- **반복적이거나, 데이터 처리에 오랜 시간이 걸리는 경우, 웹 서버 프로그램만으로는 한계가 있다.**
  - 타임아웃, UX 저하, 데드락, ...
- 배치 프로그램은 **반복적인 작업을 자동화**하고, **데이터 처리를 뒷단에서 수행**하여, 이런 문제를 해결한다.
  - 매일 정해진 시각에 고객에게 자동으로 정산한다.
  - 조회에 10초 이상 걸리는 데이터를 미리 적재하여, 실시간 트래픽을 빠르게 유지한다.

배치 프레임워크의 필요성

- 배치 작업은 보통 시간이 오래 걸리기 때문에 **작업 실패에 대한 위험이 크다.**
  - 프로그램이 멈추면 일부 고객만 정산에 실패한다.
  - 데이터를 적재하다 실패하여, 일관성이 망가진 데이터를 조회한다.
- 또한 **실패한 작업을 재시도하는 것 역시 다중 실행에 대한 위험이 크다.**
  - 정산 배치를 다시 돌리다가, 정산이 완료된 일부 고객은 2번 정산된다.
- 배치 프레임워크는 실패를 상정하고, 이를 **안전하게 재시도하는 기반을 마련한다.**
  - 어떤 고객이 정산에 실패했는지 추적하며, 이를 바탕으로 안전하게 재시도한다.

Spring Batch의 주요 컴포넌트

예시

- transaction 테이블에는 거래 정보가 담겨있다.
- 매일 이 테이블을 읽어서 거래를 집계하고 대상 테이블에 적재한다.
- 대상 테이블은 day_settlement 테이블이다.
- 100명의 고객이 존재한다.

Spring Batch의 주요 컴포넌트

Job

: 작업에 대한 로직 그 자체

일정산을 예시로 들면,

```
void job() {
  var data = Table_Transaction.read();
  var aggregatedData = Aggregate.of(data);
  Table_DaySettlement.write(aggregatedData);
}
```

Spring Batch의 주요 컴포넌트

JobInstance

: 해당 Job을 실제로 실행해야 하는 대상

- 02/27에 대한 정산, 02/28에 대한 정산, ...
  - 각각이 JobInstance이다.

일정산 작업 : Job

02/27의 일정산 작업 : JobInstance

02/28의 일정산 작업 : JobInstance

Spring Batch의 주요 컴포넌트

JobParameter

: JobInstance를 식별하는 값이다.

- 예시에서는 날짜 : 02/27, 02/28, ...

작업 자체인 Job에 식별자인 JobParameter를 붙이면 JobInstance가 된다.

일정산 작업 : Job

02/28의 일정산 작업 : JobInstance

02/28 : JobExecution

Spring Batch의 주요 컴포넌트

JobExecution

: JobInstance에 대한 실제 시도

- 02/28 정산이 한번에 성공했다면 1개의 JobExecution이 존재한다.
- 하지만 2번 실패하고 성공했다면 3개가 존재한다.

일정산 작업 : Job

02/28의 일정산 작업 : JobInstance

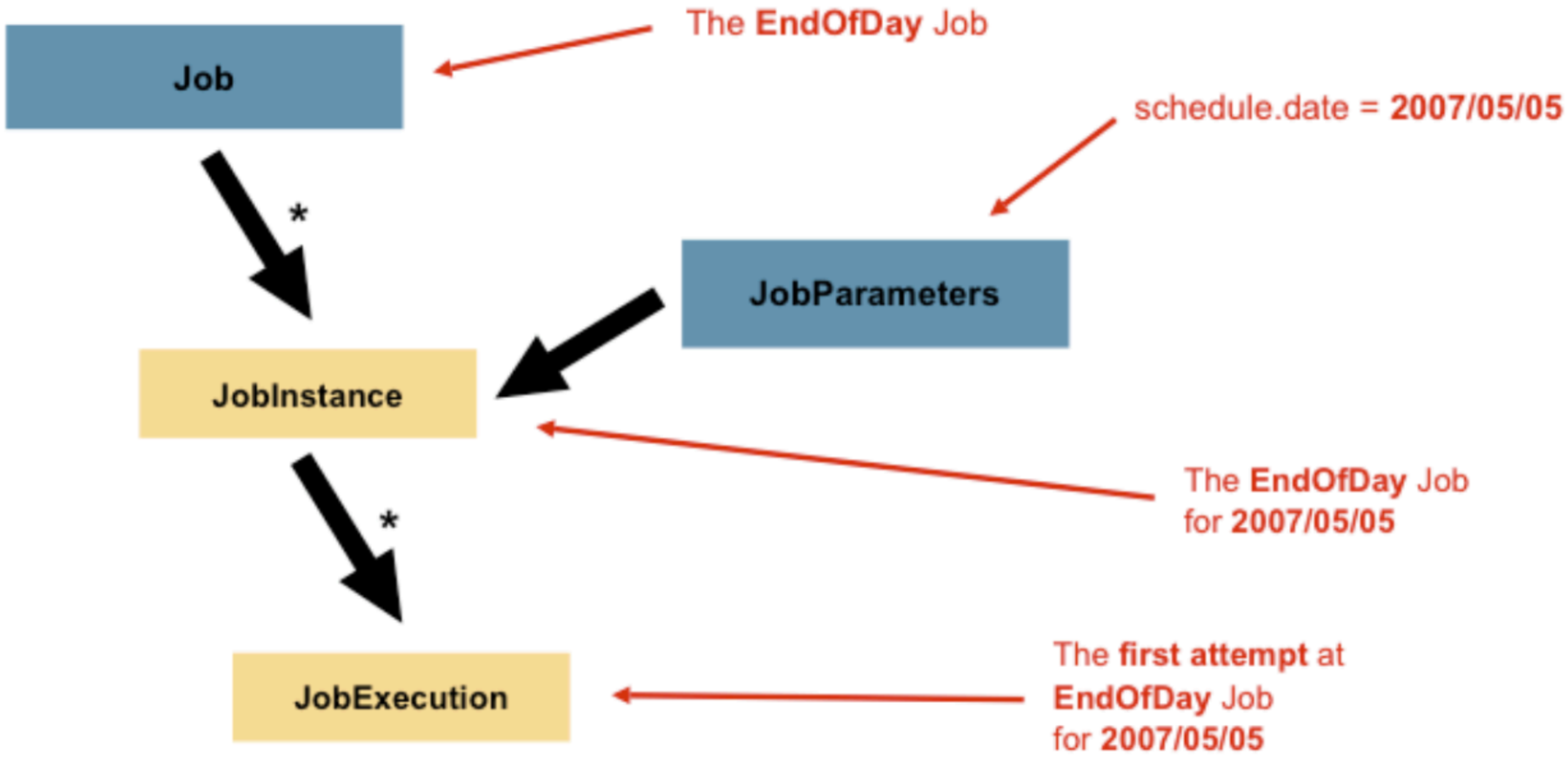
02/28 : JobExecution

02/28의 일정산 작업에 대한 첫번재 시도 : JobExecution

02/28의 일정산 작업에 대한 두번재 시도 : JobExecution

Spring Batch의 주요 컴포넌트

Job, JobInstance, JobParameter, JobExecution



Spring Batch의 주요 컴포넌트

Job, JobInstance, JobParameter, JobExecution

실제로 자주 다루게 되는 것은 Job / JobParameter 정도다.

- Job은 비즈니스 로직이 담기기 때문에 열심히 작성해야 하고,
- JobParameter는 실제로 배치잡을 실행할 때 넘겨줘야 하기 때문이다.

나머지 부분은 프레임워크가 많이 캡슐화한다.

Job을 구성해보자

Step

: Job을 구성하는 **독립적인** 단계

- Job은 여러 단계(Step)으로 구성된다.
- 각 Step은 다른 Step과 데이터를 공유하지 않는다.
  - 읽기/쓰기 작업이 모두 1개 Step 안에서 발생한다.

Job을 구성해보자

예시

- transaction 테이블에는 거래 정보가 담겨있다.
- 매일 이 테이블을 읽어서 거래를 집계하고 대상 테이블에 적재한다.
- 대상 테이블은 day_settlement 테이블이다.
- 100명의 고객이 존재한다.

→

1. transaction 테이블에서 데이터 읽기
2. 고객 번호별로 집계하기
3. day_settlement 테이블에 데이터 쓰기

Job을 구성해보자

예시

1. transaction 테이블에서 데이터 읽기
2. 고객 번호별로 집계하기
3. day_settlement 테이블에 데이터 쓰기

- 읽고 처리하고 쓰는 단위가 1개이다.
- **따라서 1개 Step으로 구성하면 된다.**

Step을 구성해보자

아래처럼 작성하면 될까?

```
void step(LocalDate today) {
  var data = Table_Transaction.read(today, 1, 100);
  var aggregatedData = Aggregate.of(data);
  Table_DaySettlement.write(today, aggregatedData);
}
```

Step을 구성해보자

Tasklet

놀랍게도 맞다. 이를 Tasklet이라 부른다.

```
@Component
public class SettlementTasklet implements Tasklet {

    private final TransactionRepository transactionRepository;
    private final SettlementRepository settlementRepository;
    private final LocalDate today;

    @Override
    public RepeatStatus execute() {
        var data = transactionRepository.read(today, 1, 100);
        var aggregatedData = Aggregate.of(data);
        settlementRepository.write(today, aggregatedData);
        return RepeatStatus.FINISHED;
    }
}
```

Step을 구성해보자

Tasklet

: 아주 간단한 작업 단계

- 한번에 쫙 돈다. (고객번호 1~100)
- 따라서 1개 JobExecution에 대해서 1번 실행된다.

Step을 구성해보자

Tasklet의 문제점

- Tasklet은 간단하지만, 중간에 실패하는 경우 처음부터 재실행해야 한다.
- 즉, 재실행시 다루는 데이터의 크기 자체가 매우 크다.
  → **재실행이 매우 오래 걸린다.**

1~100에 대해 예외 발생, 전체 롤백

1~100에 대해 예외 발생, 전체 롤백

1~100에 대해 성공

...

Step을 구성해보자

청크 지향 Step

: 데이터를 쪼개서 여러번에 걸쳐 실행한다.

- 따라서 재실행에 유리하다.
- 2번 청크까지 성공했고, 3번 청크에서 실패했다면 3번부터 다시 실행할 수 있다.

1~10에 대해 성공

11~20에 대해 실패, 해당 청크만 롤백

11~20에 대해 성공

21~30에 대해 성공

...

Step을 구성해보자

청크 지향 Step - Reader/Processor/Writer

: 청크 지향 Step은 복잡한 상황을 전제하기 때문에, Reader/Processor/Writer 로 나뉜다.

- Reader : 데이터를 읽는 컴포넌트
  - 1개씩 계속 읽는다.
- Processor : 데이터를 가공하는 컴포넌트
  - 1개씩 계속 가공한다.
  - 0..N 개 구성 가능하다.
- Writer : 데이터를 쓰는 컴포넌트
  - 청크 단위로 쓴다.

대상 데이터가 100개, 청크가 10이라면

Reader는 100번 실행된다. (1개씩 100번)

Processor는 100번 실행된다. (1개씩 100번)

Writer는 10번 실행된다. (10개씩 10번)

Step을 구성해보자

청크 지향 Step - Reader/Processor/Writer

```
public class DailySettlementStepConfig {

    public ItemReader<Data> reader() {
        return () -> Table_Transaction.readNext();
    }

    public ItemProcessor<Data, AggregatedData> processor() {
        return data -> Aggregate.of(data);
    }

    public ItemWriter<AggregatedData> writer() {
        // List<AggregatedData>
        return datas -> Table_DaySettlement.write(datas);
    }

    public Step dailySettlementStep() {
        return new StepBuilder()
                .<Data, AggregatedData>chunk(10, transactionManager)
                .reader(reader())
                .processor(processor())
                .writer(writer())
                .build();
    }
}
```

Job/Step을 구성해보자

정리

- Job은 작업 그 자체이다.
  - Job은 여러 Step으로 구성된다.
- Step은 작업의 하위 단계들이다. (현재 예시는 간단해서 1개 Step 밖에 없었다.)
- Step의 종류는 한번에 쭉 도는 Tasklet과 여러 청크로 나눠 도는 Chunk 지향 Step이 있다.
  - Tasklet은 실패 후 재시도 시, 다시 처음부터 해야 하므로 시간/메모리 낭비가 심하다.
  - Chunk 지향 Step은 실패 후 재시도 시, 실패 청크부터 시작하므로 비교적 최적화된다.

여러 Step을 구성하는 상황

- 이제 월정산을 한다.
  - 월말에 고객 100명에 대해 1~31일 정산을 하면 된다.
- 하지만 국세청의 감사 때문에 일주일 단위 정산데이터도 준비해야 한다.

여러 Step을 구성하는 상황

1개 Step을 활용하면

- 월정산 Job
  - 월정산 Step (청크 지향)
    - Reader : transaction 테이블 읽기
    - Processor : 집계하기
    - Writer : month_settlement 테이블 쓰기
- 국세청 Job
  - 주정산 Step (청크 지향)
    - Reader : transaction 테이블 읽기
    - Processor : 집계하기
    - Writer : week_settlement 테이블 쓰기

transaction 테이블은 거대하다.
2번 읽으면 병목이 된다.

여러 Step을 구성하는 상황

여러 Step을 활용하면

- 월정산/국세청 Job
  - 주정산 Step (청크 지향)
    - Reader : transaction 테이블 읽기
    - Processor : 집계하기
    - Writer : week_settlement 테이블 쓰기
  - 월정산 Step (청크 지향)
    - Reader : week_settlement 테이블 읽기
    - Processor : 집계하기
    - Writer : month_settlement 테이블 쓰기

transaction 테이블을 한번만 읽으므로
최적화된다.

정리

- Job : 작업 로직 자체
- Step : Job을 구성하는 각 단계
  - Tasklet : 1번 도는 간단한 Step
  - Chunk 지향 Step : 여러번 나누어 도는 복잡한 Step
    - Reader / Processor / Writer
- JobParameter : Job을 실행하는 파라미터 (날짜 등)
- JobInstance : JobParameter를 통해 특정되는 Job (일정산의 경우 날짜별로 1개씩 생성)
- JobExecution : JobInstance의 실행 결과 (여러번 시도했다면 여러개 생긴다.)

배치 프로그램을 설계해보자

- 제공된 러닝 데이터를 기반으로 매일 새벽에 러닝 코스를 만들어야 한다.
- 러닝 데이터는 다음 3가지 방식으로 제공된다.
  - GPX 파일
    - S3 버킷에 저장되어 있다.
    - 위도/경도/고도 정보가 포함된다.
  - KML 파일
    - S3 버킷에 저장되어 있다.
    - 위도/경도 정보만 존재하며, 고도 정보는 없다. (단, 고도 API를 통해 계산 가능)
  - 직접 그린 데이터
    - DB의 user_courses 테이블에 저장되어 있다.
    - 위도/경도 정보만 존재하며, 고도 정보는 없다. (단, 고도 API를 통해 계산 가능)
- 위도/경도/고도 데이터를 기반으로 도메인 로직을 태워 추가 검증을 해야 한다.
- 검증 완료된 데이터는 DB의 courses 테이블에 저장한다.

배치 프로그램을 설계해보자

- KML과 user_courses 테이블에는 고도가 없는 데이터가 존재한다.
- GPX는 고도가 있는 데이터가 존재한다.

→

고도가 있는지 없는지에 대한 분기로 Job을 나눠볼 수 있다.

- 고도 없는 데이터에서 고도 API를 통해 고도를 가져오고, 임시 저장해두는 Job
- 고도 있는 데이터를 임시 저장해두는 Job
- 임시 저장된 데이터를 검증한 후에 최종 저장하는 Job

배치 프로그램을 설계해보자

- JOB : 고도없는 데이터를 pending_courses에 적재
  (새벽 2시 실행)
  - STEP1
    - Reader : KML S3 버킷을 읽는다.
    - Writer : no_elevation_pending_courses 테이블에 쓴다.
  - STEP2
    - Reader : user_courses 테이블을 읽는다.
    - Writer : no_elevation_pending_courses 테이블에 쓴다.
  - STEP3
    - Reader : no_elevation_pending_courses 테이블을 읽는다.
    - Processor : 고도 API를 통해 고도 정보를 가져온다.
    - Writer : pending_courses 테이블에 쓴다.

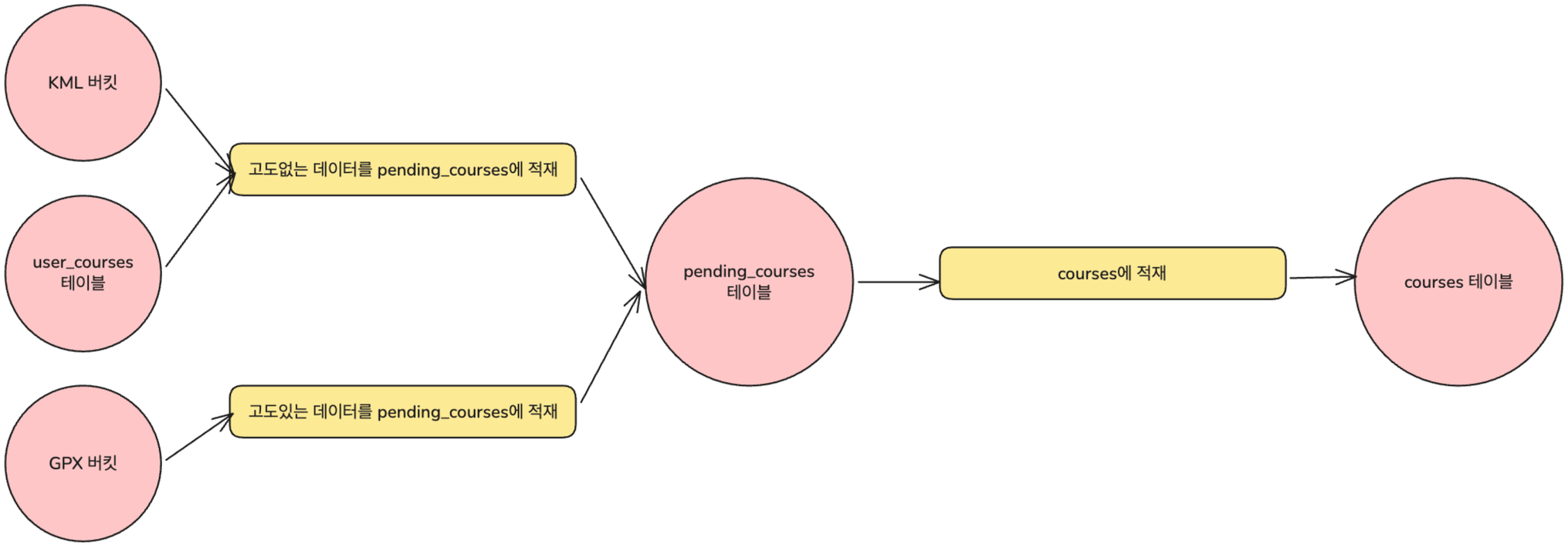
배치 프로그램을 설계해보자

- JOB : 고도있는 데이터를 pending_courses에 적재
  (새벽 2시 실행)
  - STEP1
    - Reader : GPX S3 버킷을 읽는다.
    - Writer : pending_courses 테이블에 쓴다.

배치 프로그램을 설계해보자

- JOB : courses에 적재
  (새벽 4시 실행)
  - STEP1
    - Reader : pending_courses 테이블을 읽는다.
    - Processor : 도메인 로직을 태워 검증한다.
    - Writer : courses 테이블에 쓴다.

배치 프로그램을 설계해보자



끝났습니다.
더 자세한 내용은 블로그를 참고해주세요 :)

https://dompoo.github.io/blog/스프링/배치

- 트랜잭션은 어떻게 걸리는가?
- Reader가 읽었던 데이터를 또 읽을 수 있나?
- 이전 Step이 실패하면 다음 Step도 자동으로 실패하는가?